QISKIT FALL FEST 2026

Phase 1 — Online Hackathon Participant Instructions

Qualification Stage | Starts 8 October 2026

1. How the Two-Phase Hackathon Works

Stage	What participants do	Outcome
Phase 1— Online	Choose one of 20 problem statements. Build the required solution and benchmark it on Processor A and Processor B.	Submit GitHub repository + required notebook, graphs, results and analysis.
Evaluation	Repository is checked for required files and reproducibility. LLM-assisted review and human evaluation are performed.	Final Phase-1 shortlist is announced.
Phase 2— Offline	Shortlisted teams continue with Processor C, design Processor D, and test their solution on C and D, including the required Bell-correlation/architecture study.	Final presentation, technical evaluation and final results.

PHASE 1 IN ONE SENTENCE

Choose ONE problem statement, build your solution and reproduce it on Processor A and Processor B, analyze how the two architectures affect your solution, and submit a structured GitHub repository for automated and LLM-assisted evaluation.

2. Step 1 — Choose ONE Problem Statement

You must select exactly one problem statement from the 20 available choices. The selected problem statement determines the technical task, data/instances, evaluation metrics and the starter material you receive.

Track	Code	Problem Statement
Quantum Chemistry	C1	Molecular Ground-State Energy (H_2)
Quantum Chemistry	C2	Diatomic Potential Energy Surface
Quantum Chemistry	C3	Hardware-Efficient H_2O
Quantum Chemistry	C-OI	Open Innovation — Custom Chemistry Problem
Quantum Optimization	O1	Max-Cut
Quantum Optimization	O2	Portfolio Selection
Quantum Optimization	O3	Knapsack
Quantum Optimization	O-OI	Open Innovation — Custom Optimization Problem
Quantum Simulation	S1	Ising Chain Ground State
Quantum Simulation	S2	Spin Dynamics (Real-Time)

Track	Code	Problem Statement
Quantum Simulation	S3	Noise in Trotter Simulation
Quantum Simulation	S-OI	Open Innovation — Custom Simulation Problem
Quantum Machine Learning	Q1	Biomedical Signal Classifier
Quantum Machine Learning	Q2	Quantum Kernel Classifier
Quantum Machine Learning	Q3	Noise-Resilient Classifier
Quantum Machine Learning	Q-OI	Open Innovation — Custom QML Problem
Post-Quantum Cryptography	P1	Quantum-Safe Secure Channel
Post-Quantum Cryptography	P2	Crypto Inventory & Migration Planner
Post-Quantum Cryptography	P3	PQC Performance Benchmark
Post-Quantum Cryptography	P-OI	Open Innovation — Custom PQC Problem

3. Step 2 — Look at "Starter_Guide_Phase_1" Provided in the Drive link

Drive Link: https://drive.google.com/drive/folders/1caxlXXeqjOkLmXGlimy_Etw0uPHSwyga

Starter Guide Document Link:
https://drive.google.com/drive/folders/1caxlXXeqjOkLmXGlimy_Etw0uPHSwyga

After selecting your problem statement, use the **Starter Guide** which may contain:

- Problem Statement
- Core Algorithm / Solution
- Processor A
- Processor B
- Quantum Circuit
- Transpilation & Routing
- Performance Metrics
- A/B Comparison & Architecture Analysis
- Results, Report & Submission

> **IMPORTANT**
> Do not attempt multiple problem statements. Your Phase-1 submission must clearly identify the single problem statement you selected. Open-Innovation entries are also treated as a single approved problem statement and must follow the organizer-provided framework.

4. Step 3 — Build the Core Solution

Implement the solution required by your selected problem statement. The objective is not merely to produce a final number or plot; your repository must make your work reproducible and understandable.

- Use the specified problem instance and constraints.
- Implement the required algorithm/model/workflow.

- Keep your code reproducible from a clean environment.
- Record the assumptions, parameters and important implementation choices.
- Do not change the organizer-defined benchmark in a way that makes your results incomparable with other submissions.

5. Step 4 — Run Your Solution on Processor A and Processor B

For each of the two processors, you must:

1. Run your solution using the exact processor definition supplied by the organizers.
2. Record the relevant problem-specific performance metric(s), such as energy error, objective value, accuracy, fidelity, classification performance or benchmark time.
3. Record the common architecture/resource metrics applicable to your problem, such as circuit depth, two-qubit gate count, SWAP count, qubit count, execution cost or runtime.
4. Compare Processor A and Processor B and explain how their architectural/platform differences affected your solution.
5. Support the explanation with tables, figures and measured results rather than unsupported claims.

6. What You Must Analyze

Analysis area	What to show
Problem performance	How well the solution solves the selected problem on A vs B.
Architecture / platform	How the processor/platform characteristics influence execution.
Resource usage	Relevant gate counts, depth, qubits, memory, runtime or other specified metrics.
Comparison	A clear A-versus-B comparison with evidence.
Interpretation	Why the observed differences occurred and what they mean for the solution.

> **PHASE 1 SCOPE**
> Phase 1 stops at Processor A and Processor B. Processor C and the custom Processor D are reserved for Phase 2.

7. Phase-1 Submission Package

Submit ONE GitHub repository (or the organizer-approved repository/Space format) using the exact repository structure published by the Hackathon & Competitions Cell.

Required item	Minimum expectation
main.ipynb or main.py	End-to-end implementation of the selected problem.
processors/ or equivalent	Processor A and B definitions/files supplied or required by the **Starter Guide**, plus any participant configuration needed for reproducibility.
results/	Final figures, tables and exported result files.
README.md	Problem statement selected, setup, execution steps, methodology, key results and conclusions.
Report / analysis	Notebook section or PDF containing the required analysis and interpretation.

Required item	Minimum expectation
Environment specification	requirements.txt, environment.yml, pyproject.toml or organizer-specified equivalent.
Figures / graphs	Clear plots supporting the A-vs-B comparison.

8. Repository Naming & Submission

Use the repository naming convention supplied by the organizers. At minimum, your submission form will require:

- Team/participant identification as registered.
- Selected track and problem-statement code.
- GitHub repository URL.
- Any required Hugging Face Space URL, if applicable.
- Confirmation that the repository is accessible to the judging team.
- Confirmation that the submitted version is the version to be evaluated.

9. Automated + LLM-Assisted Evaluation

After the submission deadline, the judging pipeline will begin.

Stage	What happens
Repository check	The system checks whether the repository can be accessed and whether required files/sections are present.
Reproducibility/sanity check	Required code or notebook components are checked for execution and expected outputs where technically possible.
LLM-assisted review	An LLM reviews the submission for completeness, coherence, evidence of analysis and required sections, and flags omissions/questions for human judges.
Human evaluation	Judges review the technical work, results, architecture analysis and overall quality. The LLM is an evaluation aid, not a substitute for technical judgment.

10. What the Judges Will Look For

Area	Indicative focus
Solution correctness & performance	Does the solution work? Are the results reasonable and supported by the specified reference/baseline?
A/B architecture analysis	Are the differences between A and B measured and explained using evidence?
Reproducibility	Can the judges understand and reproduce the reported results from the repository?
Technical reasoning	Does the team explain why the observed behaviour occurs?
Documentation & presentation	Are the notebook, graphs, tables and README clear and complete?

11. AI / LLM Use

AI POLICY

Participants must follow the official AI/LLM-use policy announced by the Hackathon & Competitions Cell. Any required disclosure of AI assistance must be included in the repository. Use of AI does not remove the requirement that participants understand, verify and take responsibility for their submitted work.

12. What Happens After Phase 1?

6. Submissions are evaluated after the Phase-1 deadline.
7. Finalists/shortlisted teams are announced by the organizers.
8. Only shortlisted teams proceed to the offline Phase 2.
9. The same team and selected problem statement continue into Phase 2 unless the organizers explicitly announce otherwise.
10. Phase 2 introduces Processor C and the custom Processor D design challenge.

PHASE 1 OBJECTIVE

Phase 1 objective: demonstrate that you can build, benchmark and explain a quantum/quantum-safe solution under a controlled architecture or platform comparison.

PARTICIPANT CHECKLIST BEFORE SUBMITTING

- ☐ Exactly one problem statement selected
- ☐ Core solution implemented
- ☐ Processor A run completed
- ☐ Processor B run completed
- ☐ A-vs-B comparison completed
- ☐ Required graphs/tables generated
- ☐ README completed
- ☐ Environment specification included
- ☐ Notebook/script runs from a clean setup
- ☐ Repository is accessible to judges
- ☐ All required files are present
- ☐ AI assistance disclosed if required